

RESEARCHPILOT

Investigation Report

For a fixed-size neural network trained on synthetic binary classification data with 20% randomly flipped training labels, increasing the training-set size reduces the gap between training accuracy and clean test accuracy

October 03, 2026

1 Conjecture

For a neural network with fixed architecture and parameter count, trained on synthetic binary classification samples whose training labels are randomly flipped at a rate of 20%, increasing the number of training examples reduces the gap between training accuracy and accuracy on clean test data. The claim does not specify whether this decrease is expected across repeated experiments or required for each individual experiment, nor whether the gap is signed or absolute.

Working assumptions: User-given conditions: synthetic binary classification data, fixed network size, randomly flipped training labels at a 20% rate, and clean test labels., Chosen working conditions: keep the synthetic distribution, network architecture, preprocessing, and initialization distribution unchanged as training size varies; draw test examples independently from the same distribution., For an initial numerical test, use independent label flips with probability 0.2 and measure training accuracy against the corrupted labels actually supplied during training., Use absolute gap as the primary operational meaning of 'gap', while also reporting the signed gap because the user's intended convention is unresolved., Use repeated trials to examine an average trend. Fix and report a training-budget convention before comparing sizes; equal epochs and equal update counts are different working conditions.

Unverified supporting observations to check: Not specified.

Unverified contradicting observations to check: Not specified.

2 Assessment

ResearchPilot assessment: The tested setting shows widening, not shrinking, of the absolute accuracy gap. A decrease is supported only for the signed training-minus-test difference averaged over the measured trials, and that means the accuracies move farther apart. Neither convention showed a decrease at every size within any individual trial. Expected trends and broader applicability remain unresolved. Related literature results were found, but they do not directly support or contradict the conjecture. Experimental results are inconclusive about the conjecture.

I interpret the primary claim as closer agreement between noisy-label training accuracy and clean-test accuracy, using the absolute gap. I also assess the signed difference separately. The unspecified data distribution and training procedure are not treated as a claim covering every possible setting.

Experiment 1: Training-size sweep with independent label flips provides the direct evidence. It used two-dimensional Gaussian features, clean labels determined by the first coordinate, a fixed 65-parameter tanh network, and independent training-label flips with probability 0.2. Each fit received 1200 SGD updates with batch size 32. Training accuracy was evaluated against corrupted labels. Five paired seed sweeps covered training sizes 32, 128, 512, and 2048; all 20 fits completed without censoring.

The mean absolute gaps were 0.10164, 0.16995, 0.18894, and 0.19348, respectively. Thus, the measured mean gap widened at every adjacent size, contrary to the primary interpretation. The largest-size gap exceeded the smallest-size gap in four of five seeds. Mean clean-test accuracy rose from 0.88289 to 0.98557, while mean noisy-label training accuracy changed from 0.78125 to 0.79209. More data improved clean classification without bringing the two measured accuracies closer together.

Test accuracy exceeded training accuracy in every fit. Consequently, the mean signed training-minus-test difference decreased from -0.10164 to -0.19348 . This supports a numerical decrease in the observed signed averages, not reduced disagreement. No individual seed sweep decreased at every adjacent size under either convention. The measurements therefore contradict an every-realization monotonic interpretation in this setting. Five trials without uncertainty intervals do not establish the corresponding population expectations.

The reviewed literature does not directly settle this sample-size claim. The inspected passage in [Artificial Neural Variability for Deep Learning: On Overfitting, Noise Memorization, and Catastrophic Forgetting](#) reports eventual noisy-label memorization by SGD on CIFAR-10 with 40% noise. It illustrates the relevance of training dynamics but does not vary sample size. A [Second-Order Approach to Learning with Instance-Dependent Label Noise](#) compares methods under a different noise mechanism, without measuring the specified gap. The inspected passages in [Label Noise in Adversarial Training: A Novel Perspective to Study Robust Overfitting](#) concern feature-interpolation label mismatch and epoch-wise double descent, not this sample-size comparison. Only the abstract was inspected for [Generalization Guarantees of Self-Training of Halfspaces under Label Noise Corruption](#). Its nondegradation claim concerns a particular halfspace self-training procedure, not a decreasing neural-network accuracy gap.

The central measurement limitation is target mismatch: training accuracy uses noisy labels, whereas test accuracy uses clean labels. Their difference is not solely a same-target generalization gap. The observed widening is compatible with better clean classification. Results are also conditional on equal update counts, which give larger datasets fewer expected visits per example and change full-batch training at size 32 into mini-batch training at larger sizes. Convergence was not checked. Independent flips implement the specified noise rate in expectation, rather than exactly in each dataset. Other training budgets, exact-fraction corruption, clean-label training accuracy, distributions, and architectures were not tested. These limits leave broader behavior undetermined; they do not negate the observed widening.

The conjecture is not supported as a claim that training and clean-test accuracies become closer. The recorded experiment contradicts that interpretation in its tested setting. A numerical decrease appears only for the signed training-minus-test difference averaged over the five measured seeds; it does not establish a decrease in expectation.

Experimental evidence: Experiment 1: Training-size sweep with independent label flips followed the planned comparison. It used two-dimensional Gaussian features, deterministic labels based on the first coordinate, a fixed 65-parameter tanh network, and independent flips with probability 0.2. Each fit received 1200 SGD updates with batch size 32. Nested training prefixes, initialization, and an 8192-example clean test set were shared within each seed. All

20 fits completed without censoring.

Across training sizes 32, 128, 512, and 2048, mean absolute gaps were 0.10164, 0.16995, 0.18894, and 0.19348. Thus, the observed mean gap widened at every adjacent size. Mean clean-test accuracy increased from 0.88289 to 0.98557, while noisy-label training accuracy changed from 0.78125 to 0.79209. Test accuracy exceeded training accuracy in every fit, so the mean signed difference became more negative, from -0.10164 to -0.19348 . This supports numerical signed decrease in the measured averages, not closer agreement. No individual seed sweep decreased at every adjacent size under either gap convention. The largest-size absolute gap exceeded the smallest-size gap in four of five seeds.

Related literature, not direct support or contradiction: The inspected passage in [Artificial Neural Variability for Deep Learning: On Overfitting, Noise Memorization, and Catastrophic Forgetting](#) reports eventual noisy-label memorization on CIFAR-10 with 40% noise. [A Second-Order Approach to Learning with Instance-Dependent Label Noise](#) compares methods under instance-dependent noise, in

3 Literature evidence

3.1 Supporting evidence

- None verified.

3.2 Contradictory evidence

- None verified.

3.3 Important qualifications

- None verified.

3.4 Related evidence

- While SGD with larger stochastic gradient noise memorizes noisy labels more slowly, it still memorize nearly all noisy labels at the final phase of training. [[Artificial Neural Variability for Deep Learning: On Overfitting, Noise Memorization, and Catastrophic Forgetting](#)], section-0, page 20; empirical_result. Assumptions: The figure-caption experiment uses CIFAR-10 with 40% label noise., SGD is evaluated with different learning rates through the final phase of training.. This caption reports noisy-label memorization and its dependence on training dynamics. It neither supports nor contradicts a decrease in the specified gap as sample size increases.
- PTD-R-V[40] 69.62 ± 3.35 64.73 ± 3.64 CAL 75.52 ± 3.94 70.30 ± 2.96 [[A Second-Order Approach to Learning with Instance-Dependent Label Noise](#)], section-0, page 18; empirical_result. Assumptions: The table columns correspond to noise rates $\eta = 0.2$ and $\eta = 0.4$, respectively., The comparison concerns instance-dependent label noise., Each noise rate is tested five times with a different generation matrix W ., Table 4 is labeled as a comparison without data augmentations.. The table reports higher performance for CAL than PTD-R-V at both noise rates. Sharing a 20% noise rate with the conjecture is insufficient: the noise mechanism differs, and neither sample-size effect

- We prove that the misclassification error of the resulting sequence of classifiers is bounded and show that the resulting semi-supervised approach never degrades performance compared to the classifier learned using only the initial labeled training set. [Generalization Guarantees of Self-Training of Halfspaces under Label Noise Corruption], abstract; full text not inspected; discussion. Assumptions: The abstract describes a halfspace self-training algorithm using labeled and unlabeled data., The procedure iteratively explores, pseudo-labels, retrains, and prunes examples., The comparison is against the classifier trained on the initial labeled training set.. This is an abstract-level claim of a guarantee, not a supplied proof or complete theorem statement. Its detailed mathematical assumptions are unavailable. A guarantee for this particular procedure doe
- We refer this strategy as mixup augmentation and only perform it once before the training. In this way, the true label distribution of every example in the synthetic dataset is fixed. [Label Noise in Adversarial Training: A Novel Perspective to Study Robust Overfitting], section-0, page 26; discussion. Assumptions: CIFAR-10 examples are interpolated with examples from different classes., Assigned labels are retained despite the interpolated features., Mixup augmentation is performed once before training.. This construction provides context for synthetic label mismatch, but differs from independently flipping binary training labels with probability 0.2. Fixing each example's label distribution does not
- As shown in Figure 8, different model architectures may produce slightly different double descent curves. [Label Noise in Adversarial Training: A Novel Perspective to Study Robust Overfitting], section-0, page 23; empirical_result. Assumptions: The curves concern epoch-wise double descent in adversarial training., Different architectures are compared using configurations intended to ensure comparable model capacities.. The reported curves vary training duration and architecture, not sample size at a fixed architecture. They are not a counterexample to, or evidence for, the conjectured sample-size trend.

4 Computational investigation

Experiments planned: 1. Executed successfully: 1. Stochastic trials: 0.

4.1 Experiment 1: Training-size sweep with independent label flips

Test type: falsifying. **Baselines:** Smallest training set **Metrics:** Noisy training and clean test accuracies, in proportions: $A_{s,\text{train}}(n) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{h_{\hat{\theta}_{s,n}}(X_{s,i}) = \tilde{Y}_{s,i}\}$ and $A_{s,\text{test}}(n) = \frac{1}{m} \sum_{j=1}^m \mathbf{1}\{h_{\hat{\theta}_{s,n}}(X_{s,j}^{\text{test}}) = Y_{s,j}^{\text{test}}\}$, where test size is set by `n_test`. Report individual values and arithmetic means across completed seeds., Primary absolute gap and secondary signed gap, in proportions: $G_{s,\text{absolute}}(n) = |A_{s,\text{train}}(n) - A_{s,\text{test}}(n)|$ and $G_{s,\text{signed}}(n) = A_{s,\text{train}}(n) - A_{s,\text{test}}(n)$. For each gap convention, report individual values and $\bar{G}_q(n) = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} G_{s,q}(n)$, where the completed-seed set is denoted by $\mathcal{S}$., Paired gap contrasts against the smallest-training-set baseline, in proportion differences: $D_{s,q}(n) = G_{s,q}(n) - G_{s,q}(n_0)$, with $n_0 = \min \mathcal{N}$, where the training-size sweep is denoted by $\mathcal{N}$. Report each contrast and $\bar{D}_q(n) = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} D_{s,q}(n)$ for both gap conventions., Adjacent-size gap changes, in proportion differences: $\Delta_{s,q,k} = G_{s,q}(n_{k+1}) - G_{s,q}(n_k)$, where sizes are ordered increasingly. Report individual changes and $\bar{\Delta}_{q,k} = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} \Delta_{s,q,k}$ for both gap conventions., Completion count,

in seed sweeps: $R_{\text{complete}} = |\mathcal{S}|$. If no seed sweep completes, report this count and leave accuracy, gap, and contrast aggregates unavailable.

Algorithm pseudocode:

1. Start a wall-clock timer and configure execution using `device`, `numeric_precision`, and `compute_threads`.
2. Loop over `seeds`, stopping before a new sweep if the computation deadline has been reached.
3. Initialize the specified random generator from the current seed and spawn child streams in `stream_order`, including one independent batch stream per training size.
4. Generate a training pool of length equal to the largest value in `n_train_values`: draw independent features according to $X_{s,i} \sim \mathcal{N}(0, I_d)$, with dimension from `dimension`, and assign $Y_{s,i} = \mathbf{1}\{X_{s,i,1} \geq 0\}$.
5. Generate independent corruption indicators according to $B_{s,i} \sim \text{Bernoulli}(\rho)$, with probability from `flip_probability`, and set $\tilde{Y}_{s,i} = (1 - B_{s,i})Y_{s,i} + B_{s,i}(1 - Y_{s,i})$.
6. Generate `n_test` independent feature vectors from the same feature distribution and assign clean labels using the same label rule, without corruption.
7. Draw one shared initial parameter vector using `initialization`; use the network $p_{\{\theta\}}(x) = \sigma(v^{\{\top\}} \tanh(Wx + a) + c)$, with matrix dimensions fixed by `architecture` and `dimension`.
8. Loop over `n_train_values` in increasing order and select the corresponding prefix of the training pool.
9. Reset the network to the shared initial parameter vector before each fit.
10. For every update until `updates_per_fit`, check the computation deadline and sample `batch_size` distinct training indices uniformly without replacement, independently of previous updates, using the current size's batch stream.
11. Evaluate binary cross-entropy using numerically stable logit calculations: $\ell(\theta; x, y) = -y \log p_{\theta}(x) - (1 - y) \log(1 - p_{\theta}(x))$.
12. Apply the SGD update $\theta_{t+1} = \theta_t - \frac{\eta}{|I_t|} \sum_{i \in I_t} \nabla_{\theta} \ell(\theta_t; X_{s,i}, \tilde{Y}_{s,i})$, using `learning_rate` and no other optimizer terms.
13. After the fit, check the computation deadline and predict classes on its training prefix and the shared clean test set using $h_{\theta}(x) = \mathbf{1}\{p_{\theta}(x) \geq 1/2\}$.
14. Compute the defined accuracies and gaps; retain the smallest-size fit as the baseline and compute the defined baseline contrasts for subsequent sizes.
15. After all sizes finish, compute adjacent-size changes and commit the complete seed sweep; discard the current sweep if the deadline interrupted it.
16. Aggregate the defined measurements over completed seed sweeps and return results before `max_experiment_seconds`.

Working assumptions:

- User-given conditions: synthetic binary classification data, fixed network size, randomly flipped training labels at a 20% rate, and clean test labels.
- Chosen working conditions: keep the synthetic distribution, network architecture, pre-processing, and initialization distribution unchanged as training size varies; draw test examples independently from the same distribution.
- For an initial numerical test, use independent label flips with probability 0.2 and measure training accuracy against the corrupted labels actually supplied during training.

- Use absolute gap as the primary operational meaning of 'gap', while also reporting the signed gap because the user's intended convention is unresolved.
- Use repeated trials to examine an average trend. Fix and report a training-budget convention before comparing sizes; equal epochs and equal update counts are different working conditions.
- The chosen distribution has standard normal features and a deterministic clean label given by the sign of the first coordinate.
- Nested training prefixes and shared initialization and test examples provide paired comparisons; each prefix remains an independent-example sample from the specified distribution.
- Equal update counts, rather than equal epochs, define the training-budget comparison.

Exact planned settings:

- **varied parameters:** {"n_train_values": [32, 128, 512, 2048]}
- **held constant settings:** {"dimension": 2, "feature_distribution": "standard multivariatenormal", "clean_label_rule": "first coordinate nonnegative", "flip_probability": 0.2, "n_test": 8192, "architecture": {"hidden_width": 16, "hidden_activation": "tanh", "output_activation": "sigmoid", "parameter_count": 65}, "initialization": {"weight_distribution": "independentzero-mean normal", "weight_standard_deviation": 0.1, "bias_value": 0}, "preprocessing": "none", "optimizer": "mini-batch SGD on mean binary cross-entropy", "batch_size": 32, "batch_sampling": "uniform without replacement within each update; independent across updates", "learning_rate": 0.05, "updates_per_fit": 1200, "training_budget": "equal update counts", "coupling": "nested training prefixes; shared clean test set and initialization within each seed; independent batch streams for each training size", "random_generator": "PCG64 with SeedSequencechild streams", "stream_order": ["training_features", "training_flips", "test_features", "initialization", "batches_in_training_size_order"], "device": "CPU", "numeric_precision": "float64", "compute_threads": 1}
- **baseline setting:** "smallest value in n_train_values"
- **stopping rule:** {"normal": "stop each fit after updates_per_fit updates; no early stopping or model selection", "time_limit": "check elapsed time before every update and evaluation; stop computation at compute_deadline_seconds and discard any incomplete seed sweep"}
- **work cap:** {"maximum_fits": 20, "maximum_total_updates": 24000, "compute_deadline_seconds": 270, "max_experiment_seconds": 300}
- **random seeds:** [11, 29, 47, 71, 101].

Execution: completed; run experiment_56e5d2231588; runtime 15.64 seconds.

Recorded configuration: {"varied_parameters": {"n_train_values": [32, 128, 512, 2048]}, "held_constant_settings": {"dimension": 2, "feature_distribution": "standard multivariatenormal", "clean_label_rule": "first coordinate nonnegative", "flip_probability": 0.2, "n_test": 8192, "architecture": {"hidden_width": 16, "hidden_activation": "tanh", "output_activation": "sigmoid", "parameter_count": 65}, "initialization": {"weight_distribution": "independentzero-mean normal", "weight_standard_deviation": 0.1, "bias_value": 0}, "preprocessing": "none",

```

"optimizer": "mini-batch SGD on mean binary cross-entropy", "batch_
size": 32, "batch_sampling": "uniform without replacement within
each update; independent across updates", "learning_rate": 0.05,
"updates_per_fit": 1200, "training_budget": "equal update counts",
"coupling": "nested training prefixes; shared clean test set and
initialization within each seed; independent batch streams for
each training size", "random_generator": "PCG64 with SeedSequence
child streams", "stream_order": ["training_features", "training_flips",
"test_features", "initialization", "batches_in_training_size_order"]
, "device": "CPU", "numeric_precision": "float64", "compute_threads": 1}
, "baseline_setting": "smallest value in n_train_values", "stopping_
rule": {"normal": "stop each fit after updates_per_fit updates; no early
stopping or model selection", "time_limit": "check elapsed time
before every update and evaluation; stop computation at compute_
deadline_seconds and discard any incomplete seed sweep"}, "work_
cap": {"maximum_fits": 20, "maximum_total_updates": 24000, "compute_
deadline_seconds": 270, "max_experiment_seconds": 300}}
Recorded met-
rics: {"name": "Training-size sweep with independent label flips", "n_
train_values": [32, 128, 512, 2048], "requested_seeds": [11, 29, 47, 71, 101]
, "completed_seeds": [11, 29, 47, 71, 101], "completion_count": 5, "seed_
status": [{"seed": 11, "status": "complete", "censored": false, "elapsed_
seconds": 1.9374959479999996}, {"seed": 29, "status": "complete",
"censored": false, "elapsed_seconds": 1.8213450530000008}, {"seed": 47,
"status": "complete", "censored": false, "elapsed_seconds": 1.861549536}
, {"seed": 71, "status": "complete", "censored": false, "elapsed_seconds":
1.8412309139999987}, {"seed": 101, "status": "complete", "censored":
false, "elapsed_seconds": 1.837251276}], "settings": {"dimension":
2, "feature_distribution": "standard multivariatenormal", "clean_
label_rule": "first coordinate nonnegative", "flip_probability":
0.2, "n_test": 8192, "architecture": {"hidden_width": 16, "hidden_
activation": "tanh", "output_activation": "sigmoid", "parameter_
count": 65}, "initialization": {"weight_distribution": "independent
zero-mean normal", "weight_standard_deviation": 0.1, "bias_value":
0}, "preprocessing": "none", "optimizer": "mini-batch SGD on mean
binary cross-entropy", "batch_size": 32, "batch_sampling": "uniform
without replacement within update; independent across updates",
"learning_rate": 0.05, "updates_per_fit": 1200, "training_budget": "equal
update counts", "coupling": "nested training prefixes; shared clean
test set and initialization; independent batch streams by size",
"random_generator": "PCG64 with SeedSequence child streams", "normal_
sampler": "Box-Muller with cached second variate", "stream_order":
["training_features", "training_flips", "test_features", "initializat
ion", "batches_in_training_size_order"], "device": "CPU", "numeric_
precision": "float64", "compute_threads": 1, "compute_deadline_seconds":
270, "max_experiment_seconds": 300, "maximum_fits": 20, "maximum_total_
updates": 24000, "stopping_rule": "1200 updates per fit; discard
interrupted seed sweep; no early stopping or model selection"},
"units": "accuracies and gaps are proportions; contrasts and changes
are proportion differences; BCE is nats/example", "raw_seed_sweeps":
[{"seed": 11, "fits": [{"n": 32, "train_correct": 26, "test_correct": 6766,
"train_accuracy": 0.8125, "test_accuracy": 0.825927734375, "absolute_

```

```
gap":0.013427734375,"signed_gap":-0.013427734375,"flip_count":
8,"realized_flip_fraction":0.25,"train_bce":0.5234282546638573,
"test_bce":0.3810498561076468,"last_minibatch_bce_before_update":
0.5234322740981437,"updates":1200,"absolute_baseline_contrast":
0.0,"signed_baseline_contrast":0.0},{ "n":128,"train_correct":95,
"test_correct":7559,"train_accuracy":0.7421875,"test_accuracy":
0.9227294921875,"absolute_gap":0.1805419921875,"signed_gap":-
0.1805419921875,"flip_count":31,"realized_flip_fraction":0.2421875,
"train_bce":0.5842278453468731,"test_bce":0.3788456943247961,"last_
minibatch_bce_before_update":0.5087948986575382,"updates":1200,
"absolute_baseline_contrast":0.1671142578125,"signed_baseline_
contrast":-0.1671142578125},{ "n":512,"train_correct":416,"test_
correct":8065,"train_accuracy":0.8125,"test_accuracy":0.9844970703
125,"absolute_gap":0.1719970703125,"signed_gap":-0.1719970703125,
"flip_count":90,"realized_flip_fraction":0.17578125,"train_bce":
0.5272605370781437,"test_bce":0.30046067921920067,"last_minibatch_
bce_before_update":0.6313303008194534,"updates":1200,"absolute_
baseline_contrast":0.1585693359375,"signed_baseline_contrast":-
0.1585693359375},{ "n":2048,"train_correct":1656,"test_correct":
8119,"train_accuracy":0.80859375,"test_accuracy":0.9910888671
875,"absolute_gap":0.1824951171875,"signed_gap":-0.1824951171875,
"flip_count":379,"realized_flip_fraction":0.18505859375,"train_bce":
0.5304179589434688,"test_bce":0.3012298934498729,"last_minibatch_
bce_before_update":0.40530219508559645,"updates":1200,"absolute_
baseline_contrast":0.1690673828125,"signed_baseline_contrast":
-0.1690673828125}], "adjacent_changes":[{"from_n":32,"to_n":128,
"absolute_change":0.1671142578125,"signed_change":-0.1671142578
125},{ "from_n":128,"to_n":512,"absolute_change":-0.0085449218
75,"signed_change":0.008544921875},{ "from_n":512,"to_n":2048,
"absolute_change":0.010498046875,"signed_change":-0.010498046875}]]
, {"seed":29,"fits":[{"n":32,"train_correct":25,"test_correct":7901,
"train_accuracy":0.78125,"test_accuracy":0.9644775390625,"absolute_
gap":0.1832275390625,"signed_gap":-0.1832275390625,"flip_count":6,
"realized_flip_fraction":0.1875,"train_bce":0.48712209832791636,
"test_bce":0.30375862455765856,"last_minibatch_bce_before_update":
0.487188035559736,"updates":1200,"absolute_baseline_contrast":
0.0,"signed_baseline_contrast":0.0},{ "n":128,"train_correct":102,
"test_correct":7949,"train_accuracy":0.796875,"test_accuracy":
0.9703369140625,"absolute_gap":0.1734619140625,"signed_gap":-
0.1734619140625,"flip_count":26,"realized_flip_fraction":0.203125,
"train_bce":0.5504998521044902,"test_bce":0.3541053536208181,
"last_minibatch_bce_before_update":0.49943574212702413,"updates":
1200,"absolute_baseline_contrast":-0.009765625,"signed_baseline_
contrast":0.009765625},{ "n":512,"train_correct":396,"test_correct":
8036,"train_accuracy":0.7734375,"test_accuracy":0.98095703125,
"absolute_gap":0.20751953125,"signed_gap":-0.20751953125,"flip_
count":111,"realized_flip_fraction":0.216796875,"train_bce":
0.5681856659277772,"test_bce":0.36270049067430543,"last_minibatch_
bce_before_update":0.5694287225125915,"updates":1200,"absolute_
baseline_contrast":0.0242919921875,"signed_baseline_contrast":-
0.0242919921875},{ "n":2048,"train_correct":1608,"test_correct":

```

```
8112, "train_accuracy":0.78515625, "test_accuracy":0.990234375,
"absolute_gap":0.205078125, "signed_gap":-0.205078125, "flip_count":
429, "realized_flip_fraction":0.20947265625, "train_bce":0.5553307184
417967, "test_bce":0.34722420227408934, "last_minibatch_bce_before_
update":0.531630943233359, "updates":1200, "absolute_baseline_
contrast":0.0218505859375, "signed_baseline_contrast":-0.0218505859
375]], "adjacent_changes":[{"from_n":32, "to_n":128, "absolute_
change":-0.009765625, "signed_change":0.009765625}, {"from_n":128,
"to_n":512, "absolute_change":0.0340576171875, "signed_change":-
0.0340576171875}, {"from_n":512, "to_n":2048, "absolute_change":-
0.00244140625, "signed_change":0.00244140625}]], {"seed":47, "fits":
[{"n":32, "train_correct":18, "test_correct":6292, "train_accuracy":
0.5625, "test_accuracy":0.76806640625, "absolute_gap":0.20556640625,
"signed_gap":-0.20556640625, "flip_count":7, "realized_flip_fraction":
0.21875, "train_bce":0.5869572186939424, "test_bce":0.4727084267
591161, "last_minibatch_bce_before_update":0.5869729982709911,
"updates":1200, "absolute_baseline_contrast":0.0, "signed_baseline_
contrast":0.0}, {"n":128, "train_correct":102, "test_correct":7778,
"train_accuracy":0.796875, "test_accuracy":0.949462890625, "absolute_
gap":0.152587890625, "signed_gap":-0.152587890625, "flip_count":
23, "realized_flip_fraction":0.1796875, "train_bce":0.5441951682
262888, "test_bce":0.3193521541814877, "last_minibatch_bce_before_
update":0.43190831102851257, "updates":1200, "absolute_baseline_
contrast":-0.052978515625, "signed_baseline_contrast":0.0529785156
25}, {"n":512, "train_correct":399, "test_correct":7963, "train_
accuracy":0.779296875, "test_accuracy":0.9720458984375, "absolute_
gap":0.1927490234375, "signed_gap":-0.1927490234375, "flip_count":102,
"realized_flip_fraction":0.19921875, "train_bce":0.5404925209921088,
"test_bce":0.32907570264352587, "last_minibatch_bce_before_update":
0.6447987403813326, "updates":1200, "absolute_baseline_contrast":-
0.0128173828125, "signed_baseline_contrast":0.0128173828125}, {"n":
2048, "train_correct":1616, "test_correct":7996, "train_accuracy":
0.7890625, "test_accuracy":0.97607421875, "absolute_gap":0.1870117187
5, "signed_gap":-0.18701171875, "flip_count":403, "realized_flip_
fraction":0.19677734375, "train_bce":0.5458761966352332, "test_bce":
0.33245044130750295, "last_minibatch_bce_before_update":0.5916589911
760877, "updates":1200, "absolute_baseline_contrast":-0.0185546875
, "signed_baseline_contrast":0.0185546875}], "adjacent_changes":
[{"from_n":32, "to_n":128, "absolute_change":-0.052978515625, "signed_
change":0.052978515625}, {"from_n":128, "to_n":512, "absolute_change":
0.0401611328125, "signed_change":-0.0401611328125}, {"from_n":512,
"to_n":2048, "absolute_change":-0.0057373046875, "signed_change":
0.0057373046875}]], {"seed":71, "fits":[{"n":32, "train_correct":
28, "test_correct":7821, "train_accuracy":0.875, "test_accuracy":
0.9547119140625, "absolute_gap":0.0797119140625, "signed_gap":-
0.0797119140625, "flip_count":3, "realized_flip_fraction":0.09375,
"train_bce":0.22605449197171945, "test_bce":0.14066589490853887,
"last_minibatch_bce_before_update":0.22605577225160928, "updates":
1200, "absolute_baseline_contrast":0.0, "signed_baseline_contrast":
0.0}, {"n":128, "train_correct":99, "test_correct":7860, "train_
accuracy":0.7734375, "test_accuracy":0.95947265625, "absolute_gap":
```

```
0.18603515625, "signed_gap":-0.18603515625, "flip_count":25, "realized_
flip_fraction":0.1953125, "train_bce":0.5351970404032005, "test_bce":
0.32656715122168306, "last_minibatch_bce_before_update":0.6942624761
54194, "updates":1200, "absolute_baseline_contrast":0.1063232421
875, "signed_baseline_contrast":-0.1063232421875}, {"n":512, "train_
correct":413, "test_correct":8002, "train_accuracy":0.806640625,
"test_accuracy":0.976806640625, "absolute_gap":0.170166015625,
"signed_gap":-0.170166015625, "flip_count":93, "realized_flip_
fraction":0.181640625, "train_bce":0.5325095930107014, "test_bce":
0.3086967566013028, "last_minibatch_bce_before_update":0.6078703338
015484, "updates":1200, "absolute_baseline_contrast":0.0904541015
625, "signed_baseline_contrast":-0.0904541015625}, {"n":2048, "train_
correct":1620, "test_correct":8081, "train_accuracy":0.791015625,
"test_accuracy":0.9864501953125, "absolute_gap":0.1954345703125,
"signed_gap":-0.1954345703125, "flip_count":422, "realized_flip_
fraction":0.2060546875, "train_bce":0.5619954526590829, "test_bce":
0.3329855392257084, "last_minibatch_bce_before_update":0.4257203773
9124396, "updates":1200, "absolute_baseline_contrast":0.1157226562
5, "signed_baseline_contrast":-0.11572265625}}, "adjacent_changes":
[{"from_n":32, "to_n":128, "absolute_change":0.1063232421875, "signed_
change":-0.1063232421875}, {"from_n":128, "to_n":512, "absolute_
change":-0.015869140625, "signed_change":0.015869140625}, {"from_
n":512, "to_n":2048, "absolute_change":0.0252685546875, "signed_
change":-0.0252685546875}}], {"seed":101, "fits":[{"n":32, "train_
correct":28, "test_correct":7383, "train_accuracy":0.875, "test_
accuracy":0.9012451171875, "absolute_gap":0.0262451171875, "signed_
gap":-0.0262451171875, "flip_count":6, "realized_flip_fraction":
0.1875, "train_bce":0.37939560555391083, "test_bce":0.2678012936
269427, "last_minibatch_bce_before_update":0.37940180367905824,
"updates":1200, "absolute_baseline_contrast":0.0, "signed_baseline_
contrast":0.0}, {"n":128, "train_correct":99, "test_correct":7623,
"train_accuracy":0.7734375, "test_accuracy":0.9305419921875,
"absolute_gap":0.1571044921875, "signed_gap":-0.1571044921875,
"flip_count":25, "realized_flip_fraction":0.1953125, "train_bce":
0.5206081319185282, "test_bce":0.3278415996333166, "last_minibatch_
bce_before_update":0.5652724530888353, "updates":1200, "absolute_
baseline_contrast":0.130859375, "signed_baseline_contrast":-
0.130859375}, {"n":512, "train_correct":395, "test_correct":7977,
"train_accuracy":0.771484375, "test_accuracy":0.9737548828125,
"absolute_gap":0.2022705078125, "signed_gap":-0.2022705078125,
"flip_count":110, "realized_flip_fraction":0.21484375, "train_bce":
0.566091910014586, "test_bce":0.3469702750330869, "last_minibatch_
bce_before_update":0.49724684864643853, "updates":1200, "absolute_
baseline_contrast":0.176025390625, "signed_baseline_contrast":-
0.176025390625}, {"n":2048, "train_correct":1611, "test_correct":
8061, "train_accuracy":0.78662109375, "test_accuracy":0.9840087890
625, "absolute_gap":0.1973876953125, "signed_gap":-0.1973876953125,
"flip_count":418, "realized_flip_fraction":0.2041015625, "train_bce":
0.54739485588303, "test_bce":0.33259280312773926, "last_minibatch_
bce_before_update":0.49863834218974973, "updates":1200, "absolute_
baseline_contrast":0.171142578125, "signed_baseline_contrast":-
```

```
0.171142578125}}, "adjacent_changes": [{"from_n": 32, "to_n": 128,
"absolute_change": 0.130859375, "signed_change": -0.130859375},
{"from_n": 128, "to_n": 512, "absolute_change": 0.045166015625, "signed_
change": -0.045166015625}, {"from_n": 512, "to_n": 2048, "absolute_
change": -0.0048828125, "signed_change": 0.0048828125}]], "metrics":
{"train_accuracy": {"32": [0.8125, 0.78125, 0.5625, 0.875, 0.875], "128":
[0.7421875, 0.796875, 0.796875, 0.7734375, 0.7734375], "512": [0.8125,
0.7734375, 0.779296875, 0.806640625, 0.771484375], "2048": [0.80859375,
0.78515625, 0.7890625, 0.791015625, 0.78662109375]}, "test_accuracy":
{"32": [0.825927734375, 0.9644775390625, 0.76806640625, 0.9547119140
625, 0.9012451171875], "128": [0.9227294921875, 0.9703369140625,
0.949462890625, 0.95947265625, 0.9305419921875], "512": [0.984497070
3125, 0.98095703125, 0.9720458984375, 0.976806640625, 0.9737548828125],
"2048": [0.9910888671875, 0.990234375, 0.97607421875, 0.9864501953125,
0.9840087890625]}, "absolute_gap": {"32": [0.013427734375, 0.1832275390
625, 0.20556640625, 0.0797119140625, 0.0262451171875], "128":
[0.1805419921875, 0.1734619140625, 0.152587890625, 0.18603515625,
0.1571044921875], "512": [0.1719970703125, 0.20751953125, 0.1927490234
375, 0.170166015625, 0.2022705078125], "2048": [0.1824951171875,
0.205078125, 0.18701171875, 0.1954345703125, 0.1973876953125]},
"signed_gap": {"32": [-0.013427734375, -0.1832275390625, -0.2055664062
5, -0.0797119140625, -0.0262451171875], "128": [-0.1805419921875,
-0.1734619140625, -0.152587890625, -0.18603515625, -0.1571044921
875], "512": [-0.1719970703125, -0.20751953125, -0.1927490234375,
-0.170166015625, -0.2022705078125], "2048": [-0.1824951171875, -
0.205078125, -0.18701171875, -0.1954345703125, -0.1973876953125]},
"absolute_baseline_contrast": {"32": [0.0, 0.0, 0.0, 0.0, 0.0], "128":
[0.1671142578125, -0.009765625, -0.052978515625, 0.1063232421875,
0.130859375], "512": [0.1585693359375, 0.0242919921875, -0.0128173828
125, 0.0904541015625, 0.176025390625], "2048": [0.1690673828125,
0.0218505859375, -0.0185546875, 0.11572265625, 0.171142578125]},
"signed_baseline_contrast": {"32": [0.0, 0.0, 0.0, 0.0, 0.0], "128":
[-0.1671142578125, 0.009765625, 0.052978515625, -0.1063232421
875, -0.130859375], "512": [-0.1585693359375, -0.0242919921875,
0.0128173828125, -0.0904541015625, -0.176025390625], "2048": [-
0.1690673828125, -0.0218505859375, 0.0185546875, -0.11572265625,
-0.171142578125]}}, "means": {"train_accuracy": {"32": 0.78125,
"128": 0.7765625, "512": 0.788671875, "2048": 0.79208984375}, "test_
accuracy": {"32": 0.8828857421875, "128": 0.9465087890625, "512":
0.9776123046875, "2048": 0.9855712890625}, "absolute_gap": {"32":
0.1016357421875, "128": 0.1699462890625, "512": 0.1889404296875,
"2048": 0.1934814453125}, "signed_gap": {"32": -0.1016357421875, "128":
-0.1699462890625, "512": -0.1889404296875, "2048": -0.1934814453125},
"absolute_baseline_contrast": {"32": 0.0, "128": 0.068310546875, "512":
0.0873046875, "2048": 0.091845703125}, "signed_baseline_contrast":
{"32": 0.0, "128": -0.068310546875, "512": -0.0873046875, "2048": -
0.091845703125}}, "adjacent_changes": {"absolute_change": {"32-
>128": [0.1671142578125, -0.009765625, -0.052978515625, 0.1063232421
875, 0.130859375], "128->512": [-0.008544921875, 0.0340576171875,
0.0401611328125, -0.015869140625, 0.045166015625], "512->2048":
[0.010498046875, -0.00244140625, -0.0057373046875, 0.0252685546
```

```
875,-0.0048828125]], "signed_change":{"32->128":[-0.1671142578
125,0.009765625,0.052978515625,-0.1063232421875,-0.130859375],
"128->512":[0.008544921875,-0.0340576171875,-0.0401611328125,
0.015869140625,-0.045166015625],"512->2048":[-0.010498046875,
0.00244140625,0.0057373046875,-0.0252685546875,0.0048828125]}}
,"adjacent_means":{"absolute_change":{"32->128":0.068310546875,
"128->512":0.018994140625,"512->2048":0.004541015625},"signed_
change":{"32->128":-0.068310546875,"128->512":-0.018994140625,
"512->2048":-0.004541015625}},"control":{"128":[0.013427734375,
0.1832275390625,0.20556640625,0.0797119140625,0.0262451171875],
"512":[0.013427734375,0.1832275390625,0.20556640625,0.0797119140
625,0.0262451171875],"2048":[0.013427734375,0.1832275390625,
0.20556640625,0.0797119140625,0.0262451171875]},"treatment":{"128":
[0.1805419921875,0.1734619140625,0.152587890625,0.18603515625,
0.1571044921875],"512":[0.1719970703125,0.20751953125,0.1927490234
375,0.170166015625,0.2022705078125],"2048":[0.1824951171875,
0.205078125,0.18701171875,0.1954345703125,0.1973876953125]}},
"higher_supports":false,"paired_seeds":[11,29,47,71,101],"control_
censored":{"128":[false,false,false,false,false],"512":[false,
false,false,false,false],"2048":[false,false,false,false,false]}},
"treatment_censored":{"128":[false,false,false,false,false],"512":
[false,false,false,false,false],"2048":[false,false,false,false,
false]}}, "comparisons":{"absolute":{"control":{"128":[0.013427734
375,0.1832275390625,0.20556640625,0.0797119140625,0.0262451171875]
,"512":[0.013427734375,0.1832275390625,0.20556640625,0.0797119140
625,0.0262451171875],"2048":[0.013427734375,0.1832275390625,
0.20556640625,0.0797119140625,0.0262451171875]},"treatment":{"128":
[0.1805419921875,0.1734619140625,0.152587890625,0.18603515625,
0.1571044921875],"512":[0.1719970703125,0.20751953125,0.1927490234
375,0.170166015625,0.2022705078125],"2048":[0.1824951171875,
0.205078125,0.18701171875,0.1954345703125,0.1973876953125]}},
"higher_supports":false,"paired_seeds":[11,29,47,71,101],"control_
censored":{"128":[false,false,false,false,false],"512":[false,
false,false,false,false],"2048":[false,false,false,false,false]
},"treatment_censored":{"128":[false,false,false,false,false],
"512":[false,false,false,false,false],"2048":[false,false,false,
false,false]}}, "baseline_n":32,"metric":"absolute_gap"},"signed":
{"control":{"128":[-0.013427734375,-0.1832275390625,-0.2055664062
5,-0.0797119140625,-0.0262451171875],"512":[-0.013427734375,-
0.1832275390625,-0.20556640625,-0.0797119140625,-0.0262451171
875],"2048":[-0.013427734375,-0.1832275390625,-0.2055664062
5,-0.0797119140625,-0.0262451171875]},"treatment":{"128":[-
0.1805419921875,-0.1734619140625,-0.152587890625,-0.1860351562
5,-0.1571044921875],"512":[-0.1719970703125,-0.20751953125,-
0.1927490234375,-0.170166015625,-0.2022705078125],"2048":[-
0.1824951171875,-0.205078125,-0.18701171875,-0.1954345703125,-
0.1973876953125]},"higher_supports":false,"paired_seeds":[11,29,
47,71,101],"control_censored":{"128":[false,false,false,false,
false],"512":[false,false,false,false,false],"2048":[false,false,
false,false,false]},"treatment_censored":{"128":[false,false,
false,false,false],"512":[false,false,false,false,false],"2048":
```

```
[false, false, false, false, false]], "baseline_n":32, "metric":"signed_gap"}}, "trend_summary":{"absolute":{"strictly_decreasing_mean_at_every_adjacent_size":false, "strictly_decreasing_individual_seed_sweeps":0, "nondecreasing_adjacent_changes_per_seed":[2,1,1,2,2]}, "signed":{"strictly_decreasing_mean_at_every_adjacent_size":true, "strictly_decreasing_individual_seed_sweeps":0, "nondecreasing_adjacent_changes_per_seed":[1,2,2,1,1]}}, "work_executed":{"total_updates_executed":24000, "fits_started":20, "fits_finished":20, "updates_in_current_fit":1200, "current_size":2048}, "elapsed_seconds":9.415110008, "interpretation_notes":["Primary gap is absolute; secondary gap is noisy training minus clean test accuracy.", "Flips are independent with probability 0.2, not an exactly 20 percent subset.", "All measurements and paired arrays include only fully completed seed sweeps.", "Deadline-related censoring may be informative; seed_status also includes excluded seeds.", "A finite five-seed sweep does not establish an expectation-level or universal monotonicity claim.", "Reviewed papers do not test this exact sample-size/gap setting; their methods are not substituted for the specified SGD."]]
```

Finding (inconclusive): The conclusion depends on the unresolved meaning of 'gap'. Across training sizes 32, 128, 512, and 2048, the five-seed mean absolute gap increased from 0.10164 to 0.16995, 0.18894, and 0.19348. This contradicts the proposed reduction under the primary absolute-gap interpretation. Conversely, the mean signed training-minus-test gap decreased from -0.10164 to -0.19348, supporting a numerical decrease under the signed-average interpretation, but not closer agreement between accuracies. Mean clean test accuracy rose from 0.88289 to 0.98557, while mean noisy-label training accuracy changed from 0.78125 to 0.79209. Neither gap decreased at every adjacent size in any individual seed sweep. Thus the experiment provides informative, interpretation-dependent evidence, but does not unambiguously resolve the conjecture as stated. **Uncertainty:** Only five seed sweeps and four training sizes were measured. No statistical test or uncertainty interval was reported, so the observed mean trends do not establish population expectations. Test accuracies are finite-sample measurements on 8192 clean examples per seed. All requested sweeps completed without censoring. Float64 was used, but numerical sensitivity and optimization convergence were not checked. The unresolved signed-versus-absolute and average-versus-individual interpretations prevent **Robustness:** Seeds 11, 29, 47, 71, and 101 completed all four training sizes. The absolute gap at size 2048 exceeded its size-32 baseline in four seeds; seed 47 showed a decrease. No seed exhibited strictly decreasing gaps across all adjacent sizes under either convention. Checks were limited to one two-dimensional Gaussian generator, one 65-parameter tanh network, independent flips with probability 0.2, and 1200 SGD updates per fit. No robustness checks across architectures, data generators, noise implement **Outcome confounders:**

- Training accuracy is measured against corrupted labels while test accuracy uses clean labels. Their difference includes label-target mismatch, not solely generalization error; improved clean classification can therefore widen the absolute gap.
- Independent flips produce a 20% rate in expectation, not exactly in each sample. At size 32, realized flip fractions ranged from 0.09375 to 0.25, which can affect baseline gaps.
- Equal update counts with fixed batch size give larger datasets fewer expected visits per example. Size effects are therefore conditional on this training-budget convention rather than equal epochs or convergence.
- Size 32 uses the entire training set in every batch, whereas larger sizes use stochas-

tic subsets. Independent batch streams and this change in gradient variability may contribute to nonmonotonic individual outcomes. **Result JSON fields used:** /n_train_values, /means/absolute_gap, /means/signed_gap, /means/train_accuracy, /means/test_accuracy, /metrics/absolute_baseline_contrast/2048, /trend_summary, /seed_status, /settings, /raw_seed_sweeps **Artifacts:** result.json, visualization.svg

Exact reusable experiment and visualization code is available on the Experiments page.

```
# RESEARCHPILOT EXPERIMENT PLAN

# BASELINES AND METRICS

# ['Smallest training set']

# ['Noisy training and clean test accuracies, in proportions:  $(A_{s, \mathrm{train}}(n) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{\widehat{\theta}_{s,n}(X_{s,i}) = \widetilde{Y}_{s,i}\})$  and  $(A_{s, \mathrm{test}}(n) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{\widehat{\theta}_{s,n}(X_{s,i}) = Y_{s,i}\})$ ']

# WORKING ASSUMPTIONS

# ['User-given conditions: synthetic binary classification data, fixed network size, randomly flipped training labels at a 20% rate, and clean test labels.', 'Chosen working conditions: keep the synthet']

# ALGORITHM STEPS

# ['Start a wall-clock timer and configure execution using `device`, `numeric_precision`, and `compute_threads`.', 'Loop over `seeds`, stopping before a new sweep if the computation deadline has been re']

# SETTINGS AND RANDOM SEEDS

# {'varied_parameters': {'n_train_values': [32, 128, 512, 2048]}, 'held_constant_settings': {'dimension': 2, 'feature_distribution': 'standard multivariate normal', 'clean_label_rule': 'first coordinate']}

# [11, 29, 47, 71, 101]

# EXECUTABLE STUDY

# Self-contained, scalar float64 implementation of the specified neural-network SGD.

# No noise correction, regularization, model selection, or substitute classifier.

# Reviewed papers concern other noise/training regimes and do not establish this trend.

import time

START = time.monotonic()

import math

import json

import sys

SIZES = [32, 128, 512, 2048]

SEEDS = [11, 29, 47, 71, 101]

DEADLINE = 270.0

UPDATES = 1200

BATCH = 32
```

```
WIDTH = 16
NTEST = 8192
LR = 0.05
assert sys.float_info.mant_dig == 53
# Pure Python executes CPU scalar arithmetic on one thread; no numerical
libraries.

def check_deadline():
    if time.monotonic() - START >= DEADLINE:
        raise TimeoutError('computation deadline')

# SeedSequence's default four-word entropy pool and child spawn keys,
implemented

# directly to avoid a dependency on NumPy. Arithmetic here is unsigned
uint32.

MASK32 = (1 << 32) - 1
MASK64 = (1 << 64) - 1
MASK128 = (1 << 128) - 1

def seedsequence_words(entropy, spawn_key, count=8):
    ent = []
    while entropy:
        ent.append(entropy & MASK32)
        entropy >>= 32
    if not ent:
        ent = [0]
    if spawn_key and len(ent) < 4:
        ent += [0] * (4 - len(ent))
    ent += list(spawn_key)
    hc = 0x43b0d7e5
    def hashmix(value):
        nonlocal hc
        value = (value ^ hc) & MASK32
        hc = (hc * 0x931e8875) & MASK32
        value = (value * hc) & MASK32
        return value ^ (value >> 16)
    def mix(x, y):
        value = (0xca01f9dd * x - 0x4973f715 * y) & MASK32
        return value ^ (value >> 16)
    pool = [hashmix(ent[i] if i < len(ent) else 0) for i in range(4)]
    for src in range(4):
```

```

        for dst in range(4):
            if src != dst:
                pool[dst] = mix(pool[dst], hashmix(pool[src]))
    for value in ent[4:]:
        for dst in range(4):
            pool[dst] = mix(pool[dst], hashmix(value))
    hc = 0x8b51f9dd
    out = []
    for i in range(count):
        value = pool[i % 4] ^ hc
        hc = (hc * 0x58f38ded) & MASK32
        value = (value * hc) & MASK32
        out.append(value ^ (value >> 16))
    return out

class PCG64:
    # PCG XSL-RR 128/64 (not PCG64DXSM). Each stream has a SeedSequence
    child.
    MULT = 0x2360ed051fc65da44385df649fccf645
    def __init__(self, seed, child):
        words = seedsequence_words(seed, (child,))
        u = [words[2*i] | (words[2*i+1] << 32) for i in range(4)]
        initstate = (u[0] << 64) | u[1]
        initseq = (u[2] << 64) | u[3]
        self.inc = ((initseq << 1) | 1) & MASK128
        self.state = 0
        self.step()
        self.state = (self.state + initstate) & MASK128
        self.step()
        self.spare = None
    def step(self):
        self.state = (self.state * self.MULT + self.inc) & MASK128
    def next64(self):
        self.step()
        hi = self.state >> 64
        value = (hi ^ (self.state & MASK64)) & MASK64
        rot = hi >> 58
        return ((value >> rot) | (value << ((-rot) & 63))) & MASK64
    def uniform(self):

```

```

        return (self.next64() >> 11) * (1.0 / (1 << 53))
def normal(self):
    # Box-Muller transform of PCG64 uniforms; this intentionally does
not
    # depend on a library-specific normal sampler or its lookup
tables.
    if self.spare is not None:
        value, self.spare = self.spare, None
        return value
    u = 1.0 - self.uniform()
    angle = 2.0 * math.pi * self.uniform()
    radius = math.sqrt(-2.0 * math.log(u))
    self.spare = radius * math.sin(angle)
    return radius * math.cos(angle)
def randbelow(self, n):
    # Rejection prevents modulo bias.
    limit = (1 << 64) - ((1 << 64) % n)
    while True:
        value = self.next64()
        if value < limit:
            return value % n
def batch(self, n, k):
    # Partial Fisher-Yates shuffle via a sparse map; uniform distinct
sample.
    swaps = {}
    result = []
    for i in range(k):
        j = i + self.randbelow(n - i)
        chosen = swaps.get(j, j)
        swaps[j] = swaps.get(i, i)
        result.append(chosen)
    return result
def features(rng, n):
    result = []
    for i in range(n):
        if i % 256 == 0:
            check_deadline()
        result.append((rng.normal(), rng.normal()))
    return result

```

```

def initialize(rng):
    # W has shape (16,2); a, v have length 16; c is scalar. Total = 65.
    w0, w1 = [], []
    for j in range(WIDTH):
        w0.append(0.1 * rng.normal())
        w1.append(0.1 * rng.normal())
    return w0, w1, [0.0]*WIDTH, [0.1*rng.normal() for _ in range(WIDTH)],
0.0

def fit(initial, x, labels, n, rng, progress):
    w0, w1, a, v = [p[:] for p in initial[:4]]
    c = initial[4]
    tanh, exp, loglp = math.tanh, math.exp, math.loglp
    last_loss = None
    for step in range(UPDATES):
        check_deadline()
        indices = rng.batch(n, BATCH)
        gw0, gw1, ga, gv = ([0.0]*WIDTH for _ in range(4))
        gc, loss = 0.0, 0.0
        for i in indices:
            x0, x1 = x[i]
            y = labels[i]
            hidden = [tanh(w0[j]*x0 + w1[j]*x1 + a[j]) for j in
range(WIDTH)]
            z = c
            for j in range(WIDTH):
                z += v[j]*hidden[j]
            # BCE = softplus(z) - y*z, evaluated without overflow.
            loss += max(z, 0.0) - y*z + loglp(exp(-abs(z)))
            if z >= 0.0:
                p = 1.0 / (1.0 + exp(-z))
            else:
                ez = exp(z)
                p = ez / (1.0 + ez)
            dz = p - y
            gc += dz
            for j in range(WIDTH):
                h = hidden[j]
                dh = dz * v[j] * (1.0 - h*h)
                gv[j] += dz*h

```

```

        ga[j] += dh
        gw0[j] += dh*x0
        gw1[j] += dh*x1
    rate = LR / BATCH
    # All gradients used the old parameter vector, then one SGD
update.
    for j in range(WIDTH):
        w0[j] -= rate*gw0[j]
        w1[j] -= rate*gw1[j]
        a[j] -= rate*ga[j]
        v[j] -= rate*gv[j]
    c -= rate*gc
    last_loss = loss / BATCH
    progress['total_updates_executed'] += 1
    progress['updates_in_current_fit'] = step + 1
    return (w0, w1, a, v, c), last_loss

def evaluate(parameters, x, labels, n):
    check_deadline()
    w0, w1, a, v, c = parameters
    correct = 0
    loss = 0.0
    tanh = math.tanh
    for i in range(n):
        if i % 256 == 0:
            check_deadline()
        x0, x1 = x[i]
        z = c
        for j in range(WIDTH):
            z += v[j] * tanh(w0[j]*x0 + w1[j]*x1 + a[j])
        correct += int(int(z >= 0.0) == labels[i])
        loss += max(z, 0.0) - labels[i]*z + math.log1p(math.exp(-abs(z)))
    check_deadline()
    return correct, correct / n, loss / n

settings = {
    'dimension': 2, 'feature_distribution': 'standard multivariate
normal',
    'clean_label_rule': 'first coordinate nonnegative', 'flip_probability':
0.2,
    'n_test': NTEST, 'architecture': {'hidden_width': 16, 'hidden_

```

```

activation': 'tanh',
    'output_activation': 'sigmoid', 'parameter_count': 65},
    'initialization': {'weight_distribution': 'independent zero-mean
normal',
    'weight_standard_deviation': 0.1, 'bias_value': 0},
    'preprocessing': 'none', 'optimizer': 'mini-batch SGD on mean binary
cross-entropy',
    'batch_size': BATCH, 'batch_sampling': 'uniform without replacement
within update; independent across updates',
    'learning_rate': LR, 'updates_per_fit': UPDATES, 'training_budget':
'equal update counts',
    'coupling': 'nested training prefixes; shared clean test set and
initialization; independent batch streams by size',
    'random_generator': 'PCG64 with SeedSequence child streams',
    'normal_sampler': 'Box-Muller with cached second variate',
    'stream_order': ['training_features', 'training_flips', 'test_
features', 'initialization', 'batches_in_training_size_order'],
    'device': 'CPU', 'numeric_precision': 'float64', 'compute_threads': 1,
    'compute_deadline_seconds': 270, 'max_experiment_seconds': 300,
    'maximum_fits': 20, 'maximum_total_updates': 24000,
    'stopping_rule': '1200 updates per fit; discard interrupted seed
sweep; no early stopping or model selection'
}
completed = []
status = []
progress = {'total_updates_executed': 0, 'fits_started': 0, 'fits_finished':
0,
    'updates_in_current_fit': 0, 'current_size': None}
stop = False
for seed in SEEDS:
    if stop or time.monotonic() - START >= DEADLINE:
        status.append({'seed': seed, 'status': 'not_started_deadline',
'censored': True})
        stop = True
        continue
    sweep_start = time.monotonic()
    progress['current_size'] = None
    progress['updates_in_current_fit'] = 0
    fits_this_seed = 0
    try:
        # Spawn in the exact specified order: four data/model children,
then

```

```

# one child per training size. No global RNG is used.
streams = [PCG64(seed, i) for i in range(4 + len(SIZES))]
pool = features(streams[0], max(SIZES))
clean = [int(x[0] >= 0.0) for x in pool]
flips = [int(streams[1].uniform() < 0.2) for _ in pool]
noisy = [y ^ b for y, b in zip(clean, flips)]
test = features(streams[2], NTEST)
test_labels = [int(x[0] >= 0.0) for x in test]
initial = initialize(streams[3])
fits = []
for k, n in enumerate(SIZES):
    check_deadline()
    progress['current_size'] = n
    progress['updates_in_current_fit'] = 0
    progress['fits_started'] += 1
    params, batch_loss = fit(initial, pool, noisy, n, streams[4+k]
, progress)
    check_deadline()
    train_correct, atrain, train_loss = evaluate(params, pool,
noisy, n)
    test_correct, atest, test_loss = evaluate(params, test, test_
labels, NTEST)
    signed = atrain - atest
    fits.append({'n': n, 'train_correct': train_correct, 'test_
correct': test_correct,
                'train_accuracy': atrain, 'test_accuracy': atest,
                'absolute_gap': abs(signed), 'signed_gap': signed,
                'flip_count': sum(flips[:n]), 'realized_flip_fraction':
sum(flips[:n])/n,
                'train_bce': train_loss, 'test_bce': test_loss,
                'last_minibatch_bce_before_update': batch_loss,
'updates': UPDATES})
    fits_this_seed += 1
    progress['fits_finished'] += 1
    for f in fits:
        for convention in ('absolute', 'signed'):
            key = convention + '_gap'
            f[convention + '_baseline_contrast'] = f[key] - fits[0]
[key]

adjacent = []
for k in range(len(SIZES)-1):

```

```

        adjacent.append({'from_n': SIZES[k], 'to_n': SIZES[k+1],
                        'absolute_change': fits[k+1]['absolute_gap'] - fits[k]
                        ['absolute_gap'],
                        'signed_change': fits[k+1]['signed_gap'] - fits[k]['signed_
gap']}))

        check_deadline()

        completed.append({'seed': seed, 'fits': fits, 'adjacent_changes':
adjacent})

        status.append({'seed': seed, 'status': 'complete', 'censored':
False,

                        'elapsed_seconds': time.monotonic()-sweep_start})

    except TimeoutError:

        status.append({'seed': seed, 'status': 'interrupted_discarded',
'censored': True,

                        'finished_fits_discarded': fits_this_seed, 'interrupted_size':
progress['current_size'],

                        'updates_in_current_fit': progress['updates_in_current_fit'],

                        'elapsed_seconds': time.monotonic()-sweep_start})

    stop = True

seed_ids = [s['seed'] for s in completed]
count = len(completed)
metric_names = ['train_accuracy', 'test_accuracy', 'absolute_gap',
'signed_gap',

                'absolute_baseline_contrast', 'signed_baseline_contrast']
metrics, means = {}, {}
for name in metric_names:

    metrics[name] = {str(n): [s['fits'][k][name] for s in completed] for
k, n in enumerate(SIZES)}

    means[name] = {str(n): (sum(v)/count if count else None) for n, v in
metrics[name].items()}

adjacent, adjacent_means = {}, {}
for convention in ('absolute', 'signed'):

    name = convention + '_change'

    adjacent[name] = {str(SIZES[k]) + '->' + str(SIZES[k+1]):

        [s['adjacent_changes'][k][name] for s in completed] for k in
range(len(SIZES)-1)}

    adjacent_means[name] = {key: (sum(v)/count if count else None) for
key, v in adjacent[name].items()}

comparisons = {}
for convention in ('absolute', 'signed'):

    key = convention + '_gap'

```

```

# Smaller treatment gap supports the conjecture under either
convention.

control = {str(n): metrics[key][str(SIZES[0])][:] for n in SIZES[1:]}
treatment = {str(n): metrics[key][str(n)][:] for n in SIZES[1:]}
flags = {str(n): [False]*count for n in SIZES[1:]}

comparisons[convention] = {'control': control, 'treatment':
treatment,

    'higher_supports': False, 'paired_seeds': seed_ids,
    'control_censored': flags, 'treatment_censored': flags,
    'baseline_n': SIZES[0], 'metric': key}

trend_summary = {}
for convention in ('absolute', 'signed'):
    name = convention + '_change'
    mean_values = list(adjacent_means[name].values())
    trend_summary[convention] = {
        'strictly_decreasing_mean_at_every_adjacent_size': all(v < 0 for
v in mean_values) if count else None,
        'strictly_decreasing_individual_seed_sweeps':
            sum(all(a[name] < 0 for a in s['adjacent_changes']) for s in
completed),
        'nondecreasing_adjacent_changes_per_seed':
            [sum(a[name] >= 0 for a in s['adjacent_changes']) for s in
completed]
    }
result = {
    'name': 'Training-size sweep with independent label flips',
    'n_train_values': SIZES, 'requested_seeds': SEEDS, 'completed_seeds':
seed_ids,
    'completion_count': count, 'seed_status': status, 'settings':
settings,
    'units': 'accuracies and gaps are proportions; contrasts and changes
are proportion differences; BCE is nats/example',
    'raw_seed_sweeps': completed, 'metrics': metrics, 'means': means,
    'adjacent_changes': adjacent, 'adjacent_means': adjacent_means,
    'control': comparisons['absolute']['control'], 'treatment':
comparisons['absolute']['treatment'],
    'higher_supports': False, 'paired_seeds': seed_ids,
    'control_censored': comparisons['absolute']['control_censored'],
    'treatment_censored': comparisons['absolute']['treatment_censored'],
    'comparisons': comparisons, 'trend_summary': trend_summary,
    'work_executed': progress, 'elapsed_seconds': time.monotonic()-START,

```

```

    'interpretation_notes': [
        'Primary gap is absolute; secondary gap is noisy training minus
        clean test accuracy.',
        'Flips are independent with probability 0.2, not an exactly 20
        percent subset.',
        'All measurements and paired arrays include only fully completed
        seed sweeps.',
        'Deadline-related censoring may be informative; seed_status also
        includes excluded seeds.',
        'A finite five-seed sweep does not establish an expectation-level
        or universal monotonicity claim.',
        'Reviewed papers do not test this exact sample-size/gap setting;
        their methods are not substituted for the specified SGD.'
    ]
}

payload = json.dumps(result, ensure_ascii=False, allow_nan=False,
separators=(',', ':')).encode('utf-8')
if len(payload) > 100000:
    raise RuntimeError('result.json exceeds size limit')
with open('result.json', 'wb') as f:
    f.write(payload)

```

Visualization code (reads result.json):

```

# Static SVG only; all measurements are read from result.json.
import json
import math
import xml.etree.ElementTree as ET

with open('result.json', encoding='utf-8') as f:
    r = json.load(f)
W, H = 1180, 1000
root = ET.Element('svg', {'xmlns': 'http://www.w3.org/2000/svg', 'width':
str(W),
    'height': str(H), 'viewBox': '0 0 %d %d' % (W, H), 'role': 'img',
    'aria-label': 'Training-size sweep: accuracies, gaps, and paired gap
    changes'})
def el(tag, attrs=None, text=None, parent=root):
    e = ET.SubElement(parent, tag, {k: str(v) for k, v in (attrs or {}).
.items()})
    if text is not None:
        e.text = str(text)
    return e
def text(x, y, value, size=12, anchor='start', color='#263238'):

```

```

        return el('text', {'x': x, 'y': y, 'font-family': 'sans-serif',
'font-size': size,
        'text-anchor': anchor, 'fill': color}, value)
def line(x1, y1, x2, y2, color='#cfd8dc', width=1, **extra):
    a = {'x1': x1, 'y1': y1, 'x2': x2, 'y2': y2, 'stroke': color,
'stroke-width': width}
    a.update(extra)
    return el('line', a)
el('title', text='Fixed 65-parameter neural network with 20% independent
training-label flips')
el('desc', text='Thin lines connect individual completed seed sweeps.
Thick lines are arithmetic means. Training accuracy uses noisy labels;
test accuracy uses clean labels. All size axes are categorical, with
equal spacing, not linear sample-size axes. No uncertainty intervals are
shown.')
el('rect', {'x': 0, 'y': 0, 'width': W, 'height': H, 'fill': '#ffffff'})
text(35, 32, 'Training size, noisy-label accuracy, and clean-test
accuracy', 21)
text(35, 56, '65 parameters | SGD: 1200 updates per fit | test size: %s |
completed sweeps: %s/%s' %
        (r['settings']['n_test'], r['completion_count'], len(r['requested_
seeds']))), 13)
text(35, 77, 'Thin lines: paired seeds; thick lines: means. Smaller gaps/
changes support a decreasing-gap claim.', 12)
colors = {'train_accuracy': '#1565c0', 'test_accuracy': '#d55e00',
        'absolute_gap': '#6alb9a', 'signed_gap': '#008577',
        'absolute_baseline_contrast': '#6alb9a', 'signed_baseline_contrast':
'#008577',
        'absolute_change': '#6alb9a', 'signed_change': '#008577'}
sizes = [str(n) for n in r['n_train_values']]
adjkeys = list(r['adjacent_changes']['absolute_change'])
def panel(index, title, keys, series, source, means, ylabel, accuracy=
False):
    col, row = index % 2, index // 2
    ox, oy = 32 + col*580, 105 + row*280
    left, right, top, bottom = ox+61, ox+535, oy+53, oy+217
    text(ox, oy+17, title, 15)
    for j, (name, label) in enumerate(series):
        xx = ox + j*265
        line(xx, oy+35, xx+22, oy+35, colors[name], 3)
        text(xx+28, oy+39, label, 11)
    values = [v for name, _ in series for key in keys for v in
source[name][key]]

```

```

    if accuracy:
        low, high = 0.0, 1.0
    elif not values:
        low, high = -0.1, 0.1
    elif all(v >= 0 for v in values) and all('contrast' not in name and
'change' not in name and 'signed' not in name for name, _ in series):
        low, high = 0.0, max(0.05, max(values)*1.12)
    else:
        extent = max(0.02, max(abs(v) for v in values)*1.12)
        low, high = -extent, extent
    def yy(v):
        return bottom - (v-low)/(high-low)*(bottom-top)
    def xx(k):
        return left + k*(right-left)/max(1, len(keys)-1)
    for k in range(5):
        value = low + k*(high-low)/4
        y = yy(value)
        line(left, y, right, y, '#e5e9ec', 1)
        text(left-7, y+4, '%.3f' % value, 10, 'end')
    if low <= 0 <= high:
        line(left, yy(0), right, yy(0), '#90a4ae', 1, **{'stroke-
dasharray': '4 3'})
        line(left, top, left, bottom, '#546e7a')
        line(left, bottom, right, bottom, '#546e7a')
        text(ox+3, top-6, ylabel, 10)
        for k, key in enumerate(keys):
            x = xx(k)
            line(x, bottom, x, bottom+4, '#546e7a')
            text(x, bottom+19, key.replace('->', ' to '), 10, 'middle')
        text((left+right)/2, bottom+39,
            'Adjacent training sizes (examples)' if '->' in keys[0] else
'Training size (examples; categorical spacing)',
            11, 'middle')
    if not r['completion_count']:
        text((left+right)/2, (top+bottom)/2, 'No completed sweep:
measurements unavailable', 12, 'middle')
    return
    for name, label in series:
        color = colors[name]
        for i, seed in enumerate(r['completed_seeds']):

```

```

        pts = [(xx(k), yy(source[name][key][i])) for k, key in
enumerate(keys)]

        e = el('polyline', {'points': ' '.join('%2f,%2f' % p for p
in pts),

            'fill': 'none', 'stroke': color, 'stroke-width': 1,
'stroke-opacity': 0.28})

        el('title', text='%s: seed %s' % (label, seed), parent=e)

        for k, (x, y) in enumerate(pts):
            e = el('circle', {'cx': x, 'cy': y, 'r': 2.2, 'fill':
color, 'fill-opacity': 0.35})

            el('title', text='%s, seed %s, %s: %6f' % (label, seed,
keys[k], source[name][keys[k]][i]), parent=e)

        pts = [(xx(k), yy(means[name][key])) for k, key in enumerate(ke
ys)]

        el('polyline', {'points': ' '.join('%2f,%2f' % p for p in pts),
'fill': 'none',

            'stroke': color, 'stroke-width': 3})

        for k, (x, y) in enumerate(pts):
            e = el('circle', {'cx': x, 'cy': y, 'r': 4, 'fill': color})

            el('title', text='Mean %s, %s: %6f' % (label, keys[k],
means[name][keys[k]]), parent=e)

panel(0, 'Accuracies against different label targets', sizes,

    [('train_accuracy', 'Noisy training'), ('test_accuracy', 'Clean
test')], r['metrics'], r['means'], 'Accuracy (proportion)', True)

panel(1, 'Primary absolute gap', sizes, [('absolute_gap', '|training -
test|')],

    r['metrics'], r['means'], 'Gap (proportion)')

panel(2, 'Secondary signed gap', sizes, [('signed_gap', 'Training -
test')],

    r['metrics'], r['means'], 'Gap (proportion)')

contrast_sizes = sizes[1:]

panel(3, 'Paired contrasts vs. smallest size: absolute', contrast_sizes,

    [('absolute_baseline_contrast', 'Absolute gap minus baseline gap')],
    r['metrics'], r['means'], 'Difference (proportion)')

panel(4, 'Paired contrasts vs. smallest size: signed', contrast_sizes,

    [('signed_baseline_contrast', 'Signed gap minus baseline gap')],
    r['metrics'], r['means'], 'Difference (proportion)')

panel(5, 'Adjacent-size changes in both gap conventions', adjkeys,

    [('absolute_change', 'Absolute gap change'), ('signed_change',
'Signed gap change')],

    r['adjacent_changes'], r['adjacent_means'], 'Difference (proportion)
')

text(35, 962, 'Nested training prefixes; independent flips with probability

```

```

0.2; shared test data and initialization within each seed.', 12)
text(35, 981, 'Only complete sweeps are plotted. Deadline-interrupted
sweeps are discarded; finite seed means do not prove an expectation-level
trend.', 11)
data = ET.tostring(root, encoding='utf-8', xml_declaration=True)
assert len(data) <= 1000000
with open('visualization.svg', 'wb') as f:
    f.write(data)

```

4.2 Potential counterexamples

- None recorded.

5 Interpretation

The evidence must be read under the recorded assumptions; no narrower conjecture was established.

6 What remains uncertain

- Does 'gap' mean the signed training-minus-test difference or its absolute magnitude? With corrupted training labels and clean test labels, test accuracy can exceed training accuracy.
- Is training accuracy evaluated against corrupted training labels or the original clean labels retained by the synthetic generator?
- Does '20% randomly flipped' mean independent flips with probability 0.2 or an exactly 20% subset selected uniformly? The latter requires a rounding convention for incompatible training sizes.
- The synthetic data generator, feature dimension, clean-label mechanism, network architecture, optimizer, regularization, initialization, and stopping criterion are unspecified.
- The training-budget convention is unspecified. Equal epochs, equal update counts, equal computation, and training to a stopping criterion can produce different comparisons.
- The claim's scope is unresolved: particular data and network settings versus all such settings, a finite-range trend versus strict monotonicity, and an average effect versus a decrease in every realization.
- Is the intended outcome closer agreement between accuracies, or merely a smaller signed training-minus-test difference? Should training accuracy use corrupted or retained clean labels?
- Does the claim concern an expected trend, monotonicity over a specified size range, or every individual realization? More independent seed sweeps and paired uncertainty estimates are needed to assess expectation-level effects.
- Does the trend persist under equal epochs, convergence-based stopping, or longer training? The current comparison cannot separate sample-size effects from per-example exposure and gradient stochasticity.

- Would uniformly selecting an exactly 20% flipped subset change the result? The current implementation guarantees 20% only in expectation, and compatible sizes or a rounding rule would be needed.
- How do clean-label training accuracy and accuracy on an independently corrupted test set behave? Those measurements could help separate target mismatch from same-target generalization.
- Does any trend generalize to other synthetic label rules, dimensions, fixed architectures, optimizers, or regularization settings? No such robustness tests were supplied.
- Are there directly applicable sample-size results for noisy-training versus clean-test accuracy gaps? The reviewed passages do not provide them, and the halfspace guarantee requires full-text examination before assessing its detailed scope.

7 Sources

- [Artificial Neural Variability for Deep Learning: On Overfitting, Noise Memorization, and Catastrophic Forgetting.](#)
- [A Second-Order Approach to Learning with Instance-Dependent Label Noise.](#)
- [Generalization Guarantees of Self-Training of Halfspaces under Label Noise Corruption.](#)
- [Label Noise in Adversarial Training: A Novel Perspective to Study Robust Overfitting.](#)

8 Usage

Live model requests: 10. Reused model responses: 0. All cache hits: 0. Tokens: 155310 input, 19586 output (174896 total).

Estimated provider cost: \$0.512919 USD estimated.

- interpretation (openai:gpt-6.1-sol): 1787 input, 1654 output tokens; completed; \$0.020114 estimated.
- ai_general_reasoning (openai:gpt-6.1-sol): 24837 input, 2397 output tokens; completed; \$0.083644 estimated.
- additional_literature_search (openai:gpt-6.1-sol): 4777 input, 99 output tokens; completed; \$0.010544 estimated.
- source_selection (openai:gpt-6.1-sol): 9864 input, 221 output tokens; completed; \$0.021938 estimated.
- evidence_extraction (openai:gpt-6.1-sol): 11682 input, 1693 output tokens; completed; \$0.040294 estimated.
- experiment_planning (openai:gpt-6.1-sol): 2940 input, 2787 output tokens; completed; \$0.030189 estimated.
- experiment_code (openai:gpt-6.1-sol): 13066 input, 7933 output tokens; completed; \$0.105462 estimated.
- experiment_execution (openai:gpt-6.1-sol): 31625 input, 842 output tokens; completed; \$0.071670 estimated.
- evidence_synthesis (openai:gpt-6.1-sol): 26936 input, 1054 output tokens; completed; \$0.064412 estimated.
- confidence_estimation (openai:gpt-6.1-sol): 27796 input, 906 output tokens; completed;

\$0.064652 estimated.